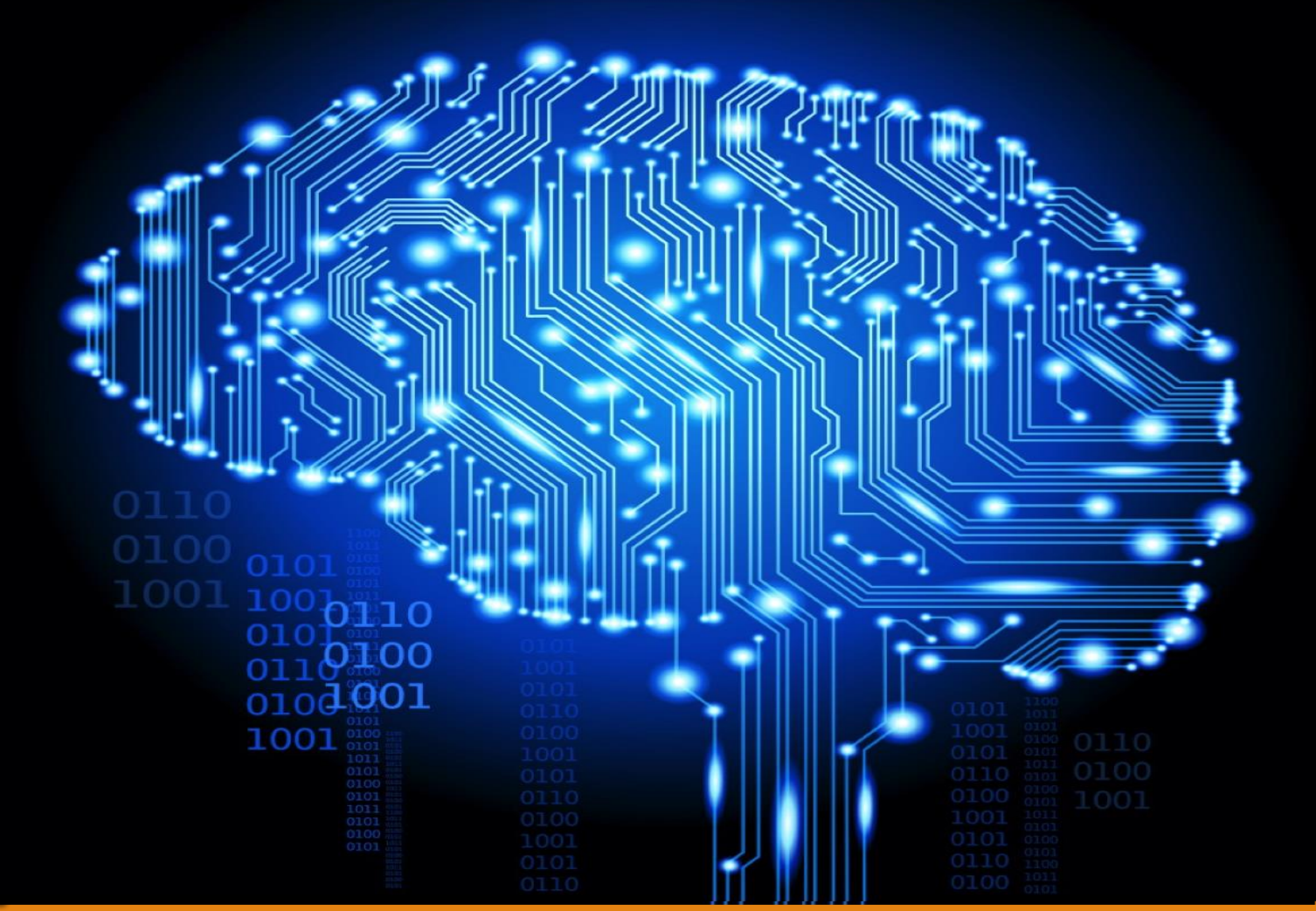
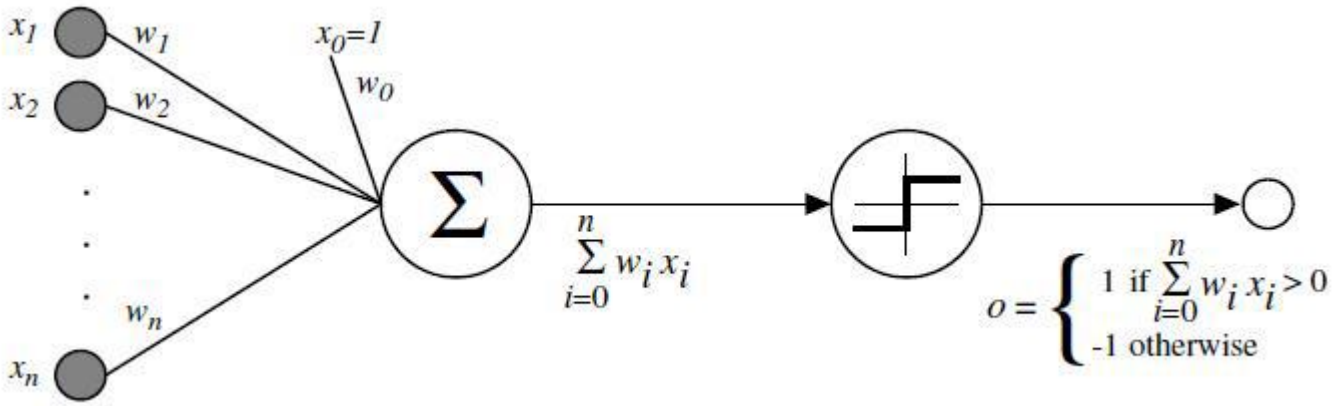


Deep Learning for Feature Extraction

Slide Courtesy : Efstratios Gavves - UVA Deep Learning

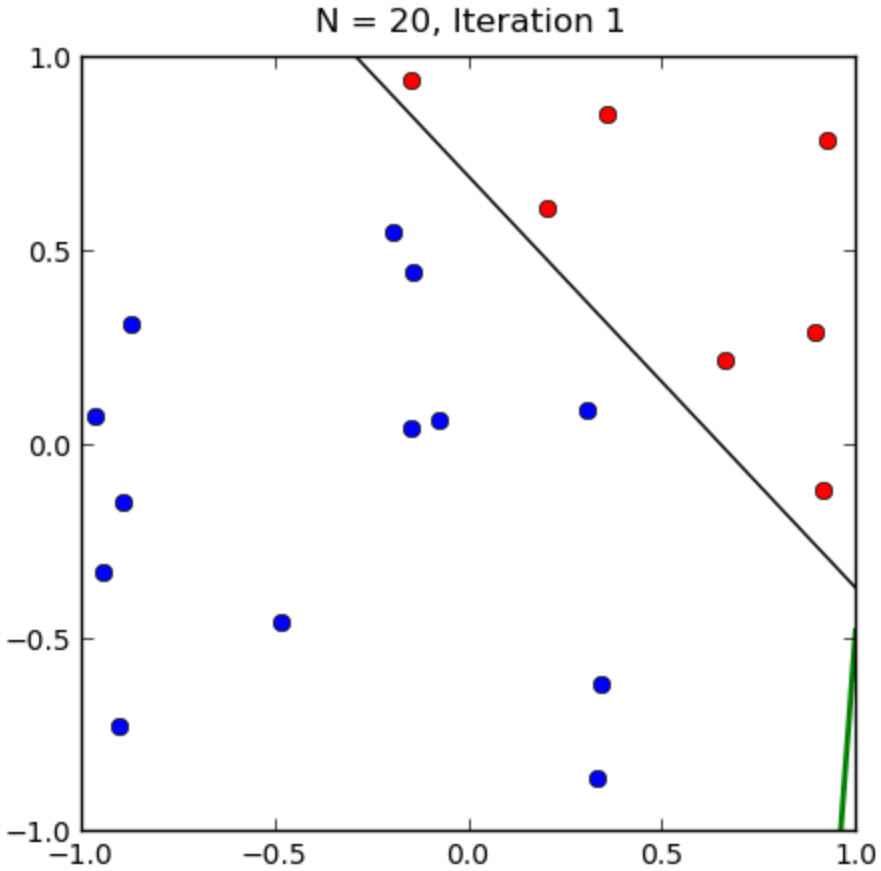
Perceptrons

- Rosenblatt proposed a machine for binary classifications
- Main idea
  - One weight w_i per input x_i
  - Multiply weights with respective inputs and add bias x_0 =+1
  - If result larger than threshold return 1, otherwise 0



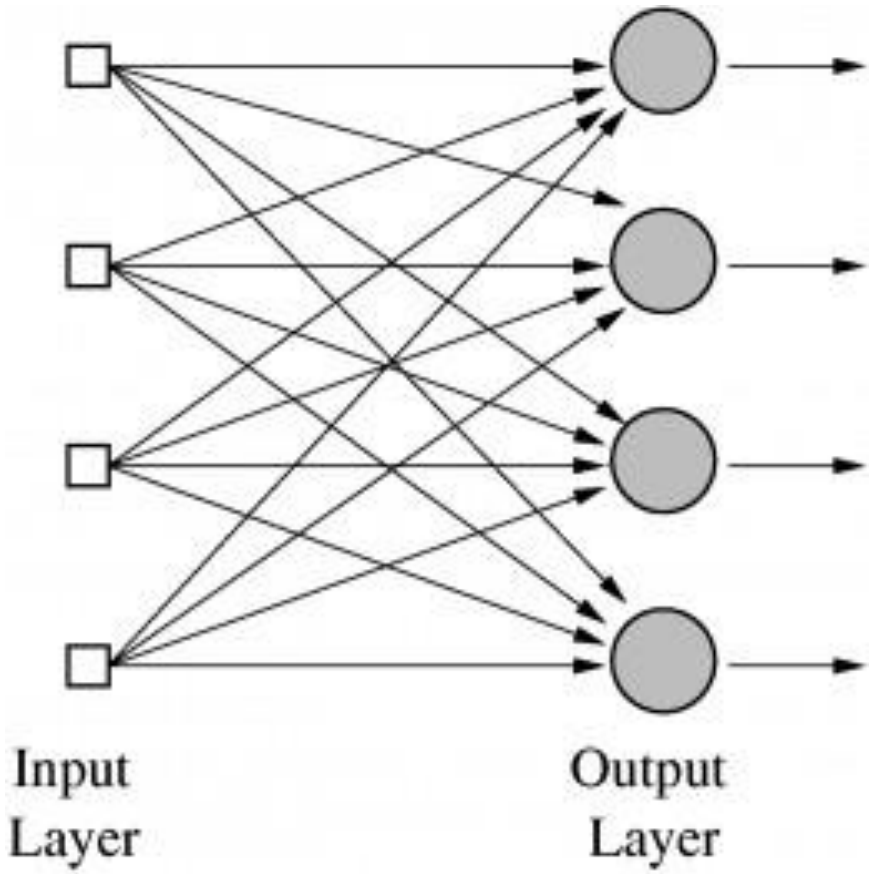
Training a perceptron

- Rosenblatt's innovation was mainly the learning algorithm for perceptrons
- Learning algorithm
  - Initialize weights randomly
  - Take one sample x_i and predict y_i
  - For erroneous predictions update weights
    - If the output was $\widehat{y_i} = 0$ and $y_i = 1$, increase weights
    - If the output was $\widehat{y_i} = 1$ and $y_i = 0$, decrease weights
  - Repeat until no errors are made

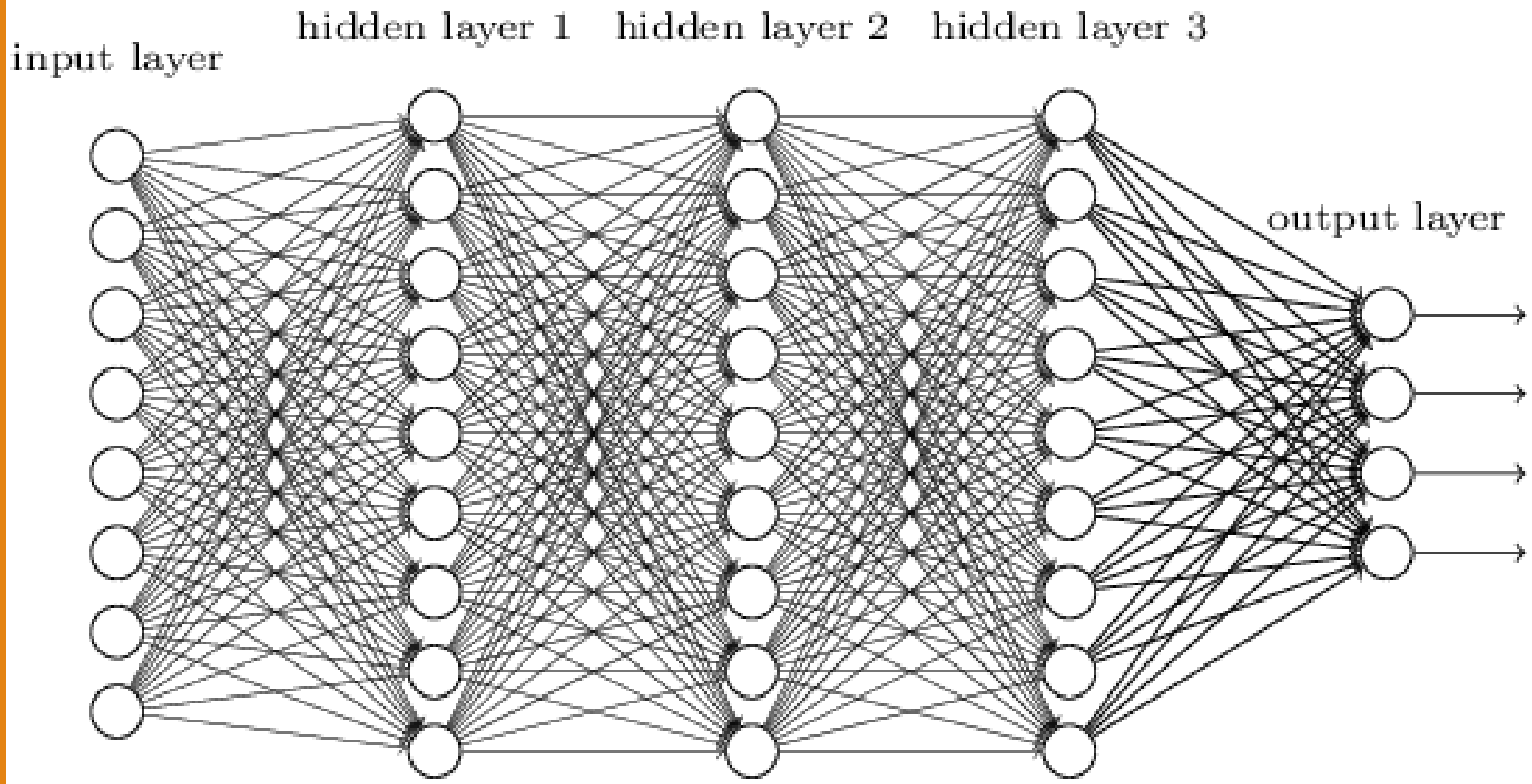


From a perceptron to a neural network

- One perceptron = one decision
- What about multiple decisions?
  - E.g. digit classification
- Stack as many outputs as the possible outcomes into a layer
  - Neural network
- Use one layer as input to the next layer
  - Multi-layer perceptron (MLP)



Deep Networks



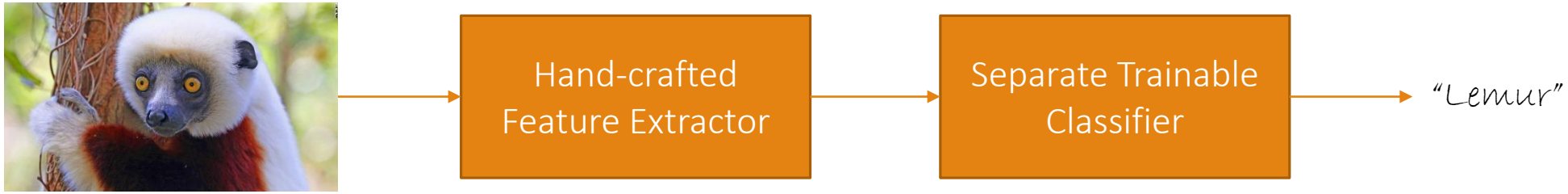
Long story short

- A family of parametric, non-linear and hierarchical representation learning functions, which are massively optimized with stochastic gradient descent to encode domain knowledge, i.e. domain invariances, stationarity.
- $a_L\left(x; \theta_{1,\ldots,\mathrm{L}}\right) = h_L\ (h_{L-1}(\ldots h_1(x, \theta_1), \theta_{L-1}),\ \theta_L)$
  - x:input, θ_l: parameters for layer l, $a_l = h_l(x, \theta_l)$: (non-)linear function
- Given training corpus $\{X, Y\}$ find optimal parameters

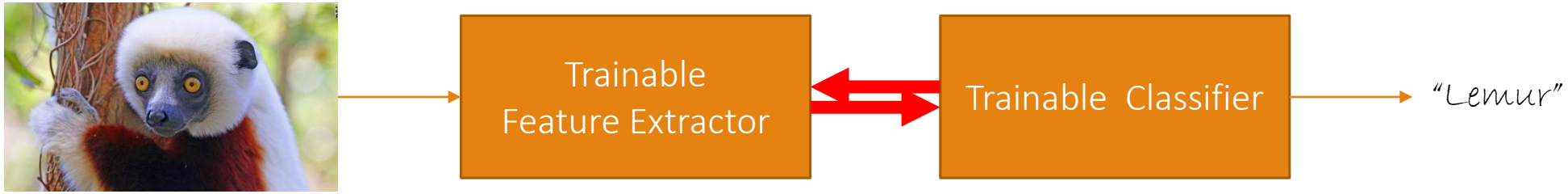
$$\theta^* \leftarrow \arg\min_\theta \sum_{(x,y)\subseteq(X,Y)} \ell(y, a_L\left(x; \theta_{1,\ldots,\mathrm{L}}\right))$$

Learning Representations & Features

- Traditional pattern recognition



- End-to-end learning → Features are also learned from data

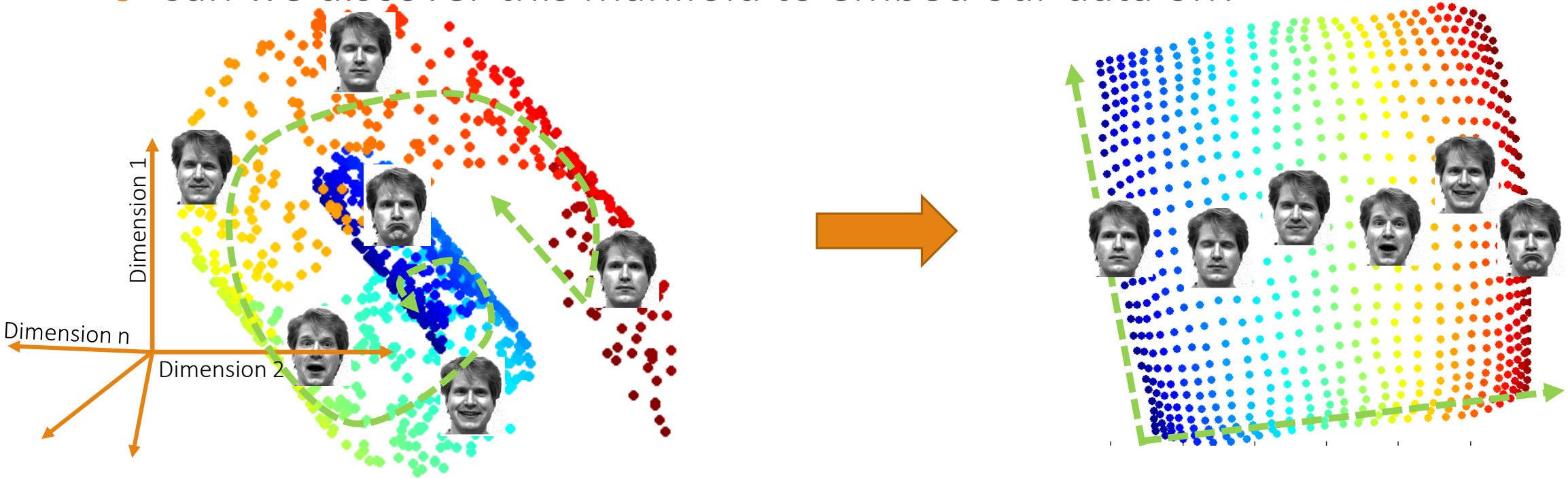


How to get good features?

- High-dimensional data (e.g. faces) lie in lower dimensional manifolds
  - **Goal:** discover these lower dimensional manifolds
  - These manifolds are most probably highly non-linear
- **Hypothesis (1):** Compute the coordinates of the input (e.g. a face image) to this non-linear manifold → data become separable
- **Hypothesis (2):** Semantically similar things lie closer together than semantically dissimilar things

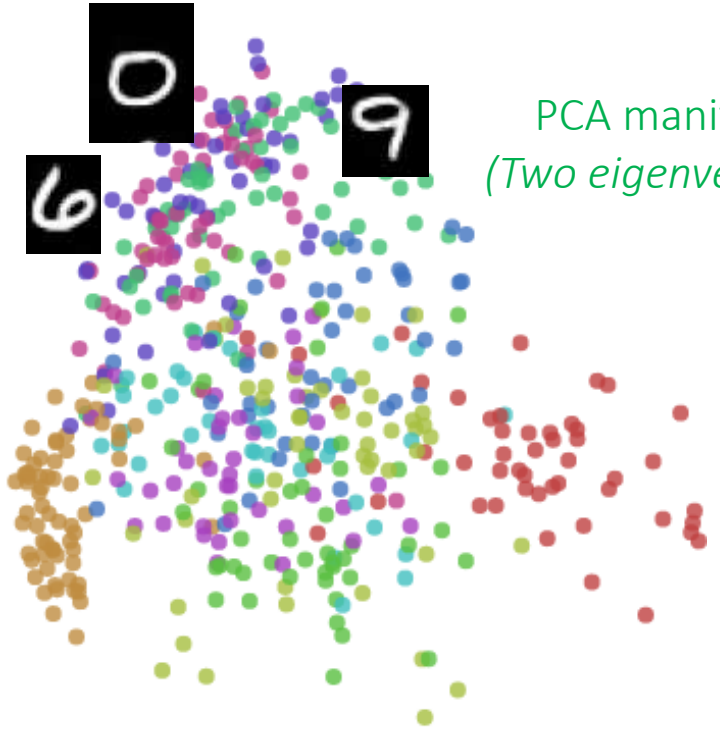
Feature manifold example

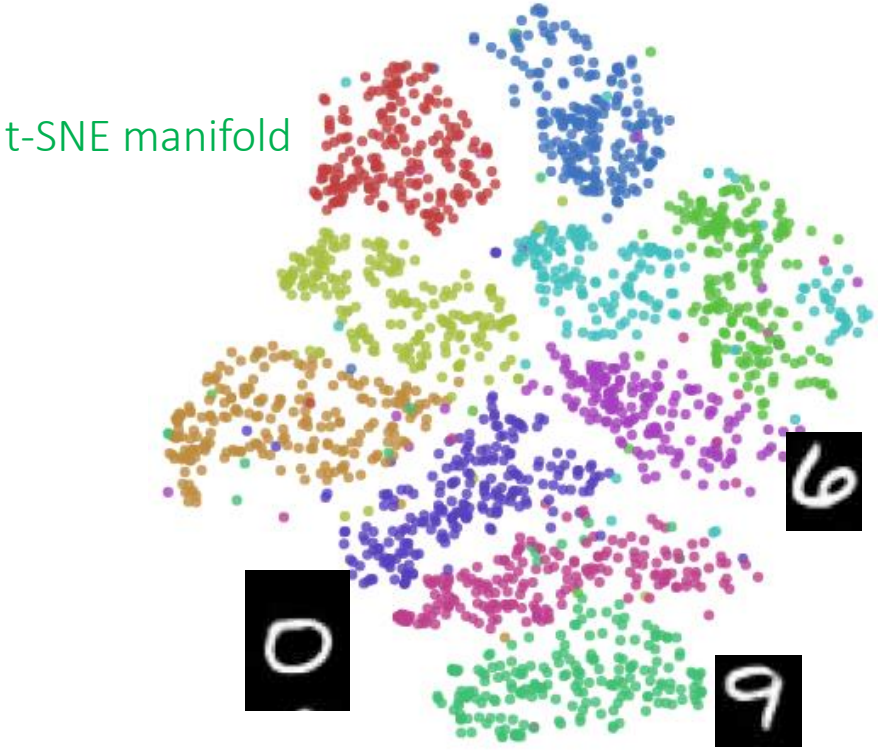
- Raw data live in huge dimensionalities
- Semantically meaningful raw data prefer lower dimensional manifolds
  - Which still live in the same huge dimensionalities
- Can we discover this manifold to embed our data on?



The digits manifolds

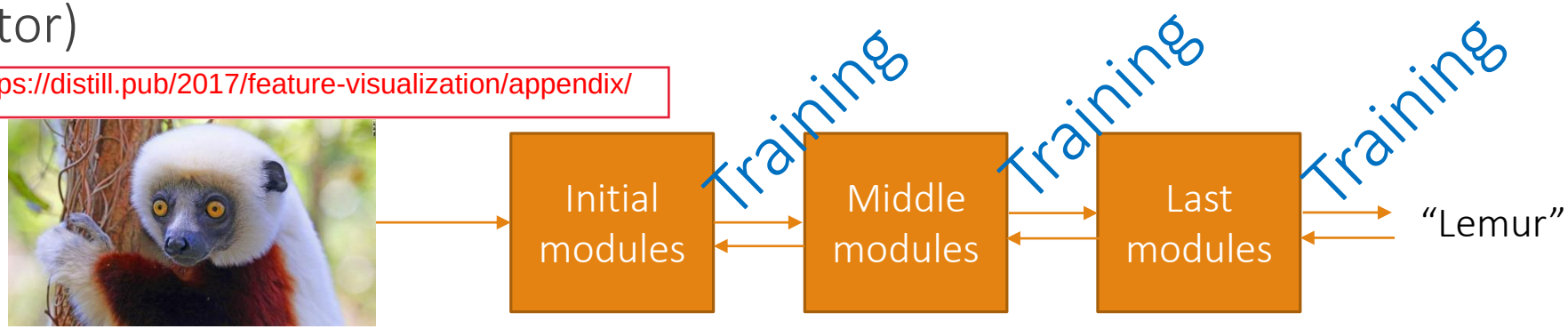
- There are good features and bad features, good manifold representations and bad manifold representations
- 28 pixels x 28 pixels = 784 dimensions





End-to-end learning of feature hierarchies

- A pipeline of successive modules
- Each module's output is the input for the next module
- Modules produce features of higher and higher abstractions
  - Initial modules capture low-level features (e.g. edges or corners)
  - Middle modules capture mid-level features (e.g. circles, squares, textures)
  - Last modules capture high level, class specific features (e.g. face detector)
  - https://distill.pub/2017/feature-visualization/appendix/



Why learn the features?

- Manually designed features
  - Often take a lot of time to come up with and implement
  - Often take a lot of time to validate
  - Often they are incomplete, as one cannot know if they are optimal for the task
- Learned features
  - Are easy to adapt
  - Very compact and specific to the task at hand
  - Given a basic architecture in mind, it is relatively easy and fast to optimize
- Time spent for <u>designing features</u> now spent for <u>designing architectures</u>

vs.